Floote Mobile App — File Inventory for Thesis Defense

This document lists **Flutter project files that ship or configure the mobile app** (Dart sources, tests, declared assets, and Android/iOS host projects). **Build caches** (for example `android/build/`, `android/.gradle/`) are generated by the toolchain and are not listed here.

Each entry gives a **concise technical role** and a **short layman’s line** you can use orally in defense.

Root — project configuration

`pubspec.yaml`

Role: Declares the Flutter package name, SDK constraint, app version, dependencies (maps, Supabase, Firebase Messaging, geolocation, HTTP, charts, image picker, navigation SDK, and others), dev tools (lints, tests, launcher icons), and asset paths.

Layman's note: This is the app’s “shopping list” of libraries and images the build bundles in.

`analysis_options.yaml`

Role: Turns on recommended Dart/Flutter lints so the codebase stays consistent and analyzable before release.

Layman's note: A spell-checker style rulebook for code quality.

`lib/` — Dart application code

`lib/main.dart`

Role: App entry: `WidgetsFlutterBinding`, Supabase initialization, Firebase Cloud Messaging hook-up after auth session changes, global `ScaffoldMessenger` key, `MaterialApp` shell, and onboarding UI that routes citizens vs rescuers (including hidden path to rescuer login).

Layman's note: Starts the app, connects to the online database, and decides which first screen you see.

`lib/firebase_options.dart`

Role: Platform-specific Firebase configuration generated by FlutterFire; used by `FcmService` to initialize push notifications.

Layman's note: Tells Firebase which project this copy of the app belongs to on each phone OS.

`lib/pages/` — full screens (routes / major UI)

`lib/pages/login_page.dart`

Role: Citizen sign-in against Supabase Auth; navigates to the main citizen home flow after success.

Layman's note: Where regular users log in.

`lib/pages/register_page.dart`

Role: Account creation and profile alignment with backend rules (identifiers, role).

Layman's note: Where new users sign up.

`lib/pages/forgot_password_page.dart`

Role: Password recovery flow integrated with Supabase Auth.

Layman's note: “Forgot password” help screen.

`lib/pages/rescuer_login_page.dart`

Role: Rescuer-specific entry; verifies role/access so responders use the correct dashboard.

Layman's note: Separate login for rescue workers.

`lib/pages/location_permission_page.dart`

Role: Guides the user through granting GPS/location access needed for SOS, reports, and routing.

Layman's note: Explains why the app needs your location.

`lib/pages/user_home_page.dart`

Role: Citizen shell: bottom navigation, tab wiring to map/home, incident reporting, emergency SOS, and profile access.

Layman's note: The main “home base” screen for ordinary users after login.

`lib/pages/home_page.dart`

Role: Central map and routing experience: destination search, hazard layers, integration with `FloodRouteService`, reroute state, and composition of split `part` files for layers, routing engine glue, and hazard UI.

Layman's note: The big map where you pick where to go and see flood-related safety on the route.

`lib/pages/emergency_page.dart`

Role: Dedicated SOS workflow for citizens: dispatch creation, status display, and emergency UX separate from flood reporting.

Layman's note: The red-alert “get help now” area of the app.

`lib/pages/incident_report_page.dart`

Role: Flood incident reporting: GPS, optional photo, duplicate-area RPC check, nearby report preview, submission to `user_reports`.

Layman's note: Where you tell the system “there is flooding here” so others can be routed safely.

`lib/pages/user_navigation_page.dart`

Role: Turn-by-turn navigation handoff using Google Navigation SDK for the citizen path, fed from computed safe routes.

Layman's note: Step-by-step driving/walking directions that respect the safer path.

`lib/pages/profile_page.dart`

Role: User profile view/edit and account-related actions tied to Supabase `profiles`.

Layman's note: Your account and settings area.

`lib/pages/rescuer_home_page.dart`

Role: Rescuer shell: navigation between rescue dashboard, map, sensor views, logs, and profile.

Layman's note: The responder’s main menu after they log in.

`lib/pages/rescuer_rescue_center_page.dart`

Role: Live SOS queue/offers for rescuers: map pins, accept/decline, integration with `sos_dispatch_offers` and dispatch lifecycle; can notify parent when a dispatch is accepted for navigation.

Layman's note: The dispatcher board where rescuers see who needs help right now.

`lib/pages/rescuer_navigation_page.dart`

Role: Rescuer navigation to incident using the Navigation SDK, aligned with accepted dispatch coordinates.

Layman's note: Directions for the rescuer to drive to the person in trouble.

`lib/pages/rescuer_sensor_network_page.dart`

Role: IoT-oriented UI: sensor nodes, water levels, connectivity status, map visualization for the “smart routing / sensor” story.

Layman's note: A dashboard of water sensors so floods are visible as data, not only as user stories.

`lib/pages/rescuer_historical_logs_page.dart`

Role: Historical or log-oriented view for rescuer operations (past activity / diagnostics depending on implementation).

Layman's note: A “what happened earlier” record for responders.

`lib/pages/` — `part` files (same library as `home_page.dart`)

These files are **`part of 'home_page.dart'`**; the compiler merges them with `home_page.dart`. They exist to keep the large home/map module readable.

`lib/pages/home_page_map_layers.dart`

Role: Map UI fragments: search bar (`TypeAheadField`), overlays, and map-adjacent widgets tied to `_HomePageState`.

Layman's note: The search bar and map decorations on top of the map.

`lib/pages/home_page_route_service.dart`

Role: Heavy routing logic: Google vs legacy fallbacks, caching, A-star / graph-style safe routing orchestration, HTTP client reuse, environment flags (`USE_GOOGLE_ROUTING`, etc.), integration with verified hazard reports.

Layman's note: The brain that keeps recalculating “the safest way there” as conditions change.

`lib/pages/home_page_hazard_widgets.dart`

Role: Hazard-related controls and displays on the home map (chips, panels, or markers depending on build).

Layman's note: The on-map warnings and controls about dangerous areas.

`lib/pages/widgets/` — reusable page widgets

`lib/pages/widgets/home_tab_widgets.dart`

Role: Shared bottom navigation and tab chrome for citizen vs rescuer variants (`HomeBottomNav` and related).

Layman's note: The bottom bar that switches between Map, Reports, SOS, etc.

`lib/pages/widgets/sos_queue_card.dart`

Role: Card UI for presenting an SOS item in a list/queue (used in rescuer or citizen contexts as wired).

Layman's note: One row/card that summarizes a single emergency ticket.

`lib/services/`

`lib/services/flood_route_service.dart`

Role: HTTP client to the FastAPI flood-routing backend; parses `FloodRouteResult` (`safe`, `rerouted`, `best_effort`, etc.), polyline and waypoint extraction for Navigation SDK, Supabase reads

for hazard context, tolerances for impassable zones.

Layman's note: Talks to the server that checks roads against floods and returns a path you can follow.

`lib/utils/` — cross-cutting helpers

`lib/utils/auth_login.dart`

Role: Normalizes login identifiers (lowercase/trim), shared user-facing error strings to avoid account enumeration and keep messaging consistent.

Layman's note: Small rules so sign-in behaves safely and predictably.

`lib/utils/fcm_service.dart`

Role: Firebase Messaging singleton: init order with Supabase, background handler registration, token lifecycle hooks for authenticated users, foreground presentation options.

Layman's note: Handles push alerts so rescuers or users get notified when something important happens.

`lib/utils/sos_dispatch_status.dart`

Role: Typed helpers/constants for SOS dispatch status values (`submitted`, `en_route`, `closed`, etc.) to match database enums.

Layman's note: Names each stage of an emergency ticket so everyone speaks the same language.

`lib/utils/sos_emergency_categories.dart`

Role: Labels/categories for SOS types (flood-related and other emergency semantics) used in UI and payloads.

Layman's note: Pick what kind of emergency it is when calling for help.

`lib/utils/sensor_reading_format.dart`

Role: Parsing/formatting helpers for sensor readings shown on rescuer IoT screens.

Layman's note: Turns raw sensor numbers into what you read on screen.

`lib/utils/philippine_phone.dart`

Role: Phone number validation or formatting tuned for Philippine numbers where the product requires it.

Layman's note: Makes sure local phone numbers look right for the country.

`test/` — automated tests

`test/widget_test.dart`

Role: Default Flutter widget smoke test scaffold; can be expanded for regression tests before defense demos.

Layman's note: A robot tap that checks the app still builds and basic UI works.

Static assets (declared in `pubspec.yaml`)

These paths are registered under `flutter: assets:` so the app can load them with `AssetImage` / `Image.asset`:

`assets/images/sos_siren.png`

Role: SOS imagery for emergency UI emphasis.

Layman's note: The siren picture on the help screen.

`assets/branding/launcher_icon.png`

Role: Source image for generated launcher icons (see `flutter_launcher_icons` in `pubspec.yaml`).

Layman's note: The app icon artwork on the home screen.

(If a path is missing locally, restore the file or run `flutter pub get` / icon generator so release builds succeed.)

`android/` — Android host project

`android/settings.gradle.kts`

Role: Gradle settings for module inclusion and plugin management.

Layman's note: Android build’s table of contents.

`android/build.gradle.kts`

Role: Top-level Android Gradle build configuration (SDK versions, shared plugins).

Layman's note: Android-wide build switches.

`android/gradle.properties`

Role: JVM memory, AndroidX, and other Gradle properties.

Layman's note: Performance and compatibility knobs for Android builds.

`android/gradle/wrapper/gradle-wrapper.properties`

Role: Pins the Gradle version for reproducible builds.

Layman's note: Locks which Android builder version everyone uses.

`android/gradlew` and `android/gradlew.bat`

Role: Gradle wrapper scripts (Unix / Windows) to build without a global Gradle install.

Layman's note: Command-line shortcuts to compile the Android side.

`android/.gitignore`

Role: Ignores local Android artifacts not meant for git.

Layman's note: Tells git to ignore machine-specific clutter.

`android/local.properties`

Role: Local SDK path on a developer machine (often gitignored).

Layman's note: Where *your* computer stores the Android SDK path.

`android/app/build.gradle.kts`

Role: App module: `applicationId`, min/target SDK, signing configs, Flutter Gradle plugin application, dependencies.

Layman's note: The Android “app package” settings and libraries.

`android/app/google-services.json`

Role: Firebase Android client configuration for FCM and Firebase services.

Layman's note: Firebase’s ID card for the Android app.

`android/app/src/main/AndroidManifest.xml`

Role: App name, launcher activity, permissions (location, internet, notifications, etc.), and Google Maps key metadata references.

Layman's note: Official list of what the Android app is allowed to do (camera, location, internet…).

`android/app/src/debug/AndroidManifest.xml`

Role: Debug-only manifest overlays (e.g., tools attributes).

Layman's note: Extra permissions or flags only while developing.

`android/app/src/profile/AndroidManifest.xml`

Role: Profile-mode manifest for performance profiling builds.

Layman's note: Settings used when profiling speed, not for store release.

`android/app/src/main/kotlin/com/example/first/MainActivity.kt`

Role: Android entry activity hosting the Flutter engine.

Layman's note: The tiny Android doorway that opens the Flutter app.

`android/app/src/main/java/io/flutter/plugins/GeneratedPluginRegistrant.java`

Role: Auto-generated plugin registration for native Android plugins.

Layman's note: Auto-wiring so map and Firebase plugins work on Android.

`android/app/src/main/res/values/google_maps_api.xml`

Role: Holds the Google Maps SDK key reference for Android map rendering.

Layman's note: Where Android finds the map license key.

`android/app/src/main/res/values/styles.xml` and `android/app/src/main/res/values-night/styles.xml`

Role: Launch theme and Material styles for light/dark.

Layman's note: Splash screen look and light vs dark styling.

`android/app/src/main/res/drawable/launch_background.xml` and `android/app/src/main/res/drawable-v21/launch_background.xml`

Role: Splash screen drawable for API level variants.

Layman's note: The first picture you see while the app loads.

`ios/` — iOS host project

`ios/Runner/AppDelegate.swift`

Role: iOS application delegate; initializes plugins and Flutter engine.

Layman's note: iOS-side startup for the Flutter app.

`ios/Runner/Info.plist`

Role: Bundle ID, display name, permission usage strings (location, notifications, etc.), URL schemes if any.

Layman's note: Apple’s checklist of app name and “why we use your location.”

`ios/Runner/Runner-Bridging-Header.h`

Role: Objective-C ↔ Swift bridging for legacy plugin headers.

Layman's note: Adapter header for mixed Apple languages.

`ios/Runner/GeneratedPluginRegistrant.h` and `ios/Runner/GeneratedPluginRegistrant.m`

Role: Generated native plugin registration.

Layman's note: Auto-wiring for iOS plugins.

`ios/Runner/Base.lproj/Main.storyboard` and `ios/Runner/Base.lproj/LaunchScreen.storyboard`

Role: Base storyboards for host window and launch screen.

Layman's note: Basic iOS window and launch layout.

`ios/Runner/Assets.xcassets/AppIcon.appiconset/Contents.json` (+ icon PNGs as present)

Role: Xcode asset catalog for application icons.

Layman's note: All the differently sized icons Apple requires.

`ios/Runner/Assets.xcassets/LaunchImage.imageset/Contents.json` (+ `README.md`)

Role: Launch image asset metadata (and Flutter readme for launch assets).

Layman's note: Startup image metadata.

`ios/RunnerTests/RunnerTests.swift`

Role: Default XCTest target stub for iOS unit/UI tests.

Layman's note: Placeholder for automated tests on iPhones.

`ios/Runner.xcodeproj/project.pbxproj`

Role: Xcode project structure: targets, build phases, file references.

Layman's note: The master blueprint Xcode opens.

`ios/Runner.xcodeproj/xcshareddata/xcschemes/Runner.xcscheme`

Role: Shared run/test/archive scheme for the Runner target.

Layman's note: How Xcode knows to “Run” the Floote app.

`ios/Runner.xcworkspace/contents.xcworkspacedata` and shared `xcshareddata` plist/settings

Role: Workspace layout for opening the project with CocoaPods/Pods integration when used.

Layman's note: Wraps the Xcode project so dependencies build together.

`ios/Flutter/Debug.xcconfig`, `ios/Flutter/Release.xcconfig`, `ios/Flutter/Generated.xcconfig`

Role: Build-time Flutter flags and generated paths for headers/frameworks.

Layman's note: iOS build recipes pointing at Flutter’s engine.

`ios/Flutter/AppFrameworkInfo.plist`

Role: Metadata for embedded Flutter framework.

Layman's note: Technical metadata for the Flutter engine bundle.

`ios/Flutter/flutter_export_environment.sh`

Role: Script exporting build environment variables for Xcode build steps.

Layman's note: Shell snippet Xcode runs so Flutter paths resolve.

`ios/Flutter/ephemeral/flutter_lldb_helper.py` and `ios/Flutter/ephemeral/flutter_lldbinit`

Role: Ephemeral debugging helpers for LLDB when debugging Flutter on iOS (regenerated by tooling).

Layman's note: Debugger helpers; safe to ignore for feature explanations.

`ios/.gitignore`

Role: Ignores Pods, build outputs, and ephemeral Flutter files as appropriate.

Layman's note: Keeps the repo clean of iOS build junk.

`web/` — optional Flutter web shell

`web/index.html`

Role: HTML host page that loads `flutter_bootstrap.js` / main.dart.js for browser builds.

Layman's note: The empty webpage wrapper if you run Floote in a browser.

`web/manifest.json`

Role: PWA-style name, colors, and icons for web install prompts.

Layman's note: Makes the web tab look like an “installable” app.

Defense sound-bite (one paragraph)

The Floote mobile codebase is organized as **Flutter Dart modules** for citizens (map, flood reports, SOS, navigation) and rescuers (dispatch center, navigation, sensor dashboard), a **single flood-routing service** talking to FastAPI, **Firebase Messaging** for pushes, and **standard Android/iOS host projects** that supply permissions, keys, and native SDK bridges. Splitting `home_page` into **`part` files** keeps the complex map and routing story maintainable while preserving one logical screen for defense walkthroughs.

PDF generation

From the repository root:

`py -3 docs/generate_thesis_pdf.py`

This script builds PDFs for this inventory and the separate thesis report when the corresponding `.md` files exist under `docs/`.